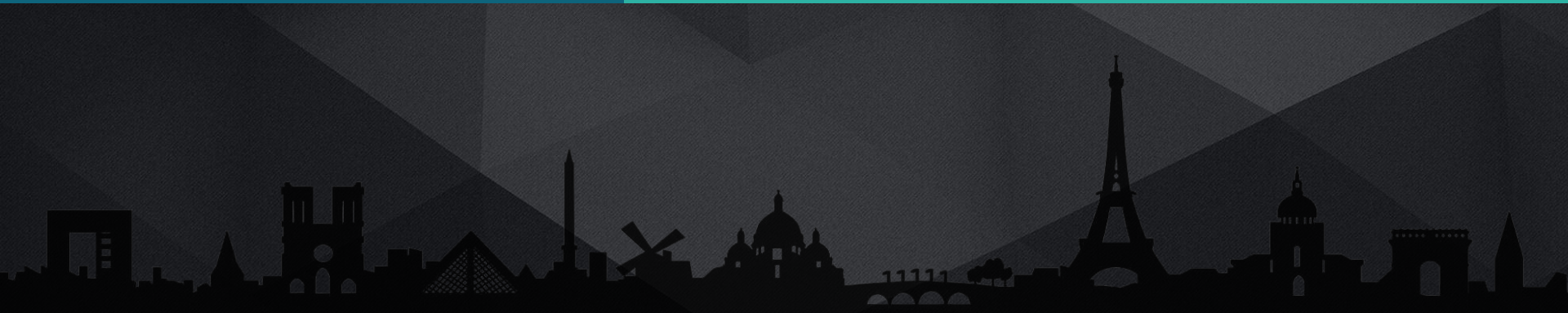


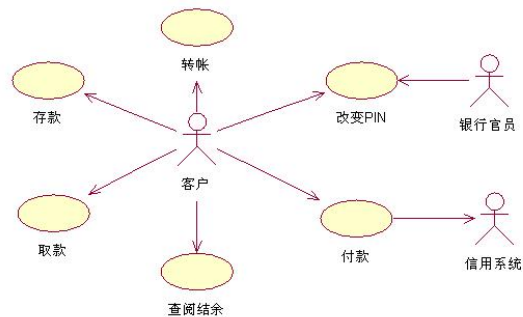
软件工程复习资料



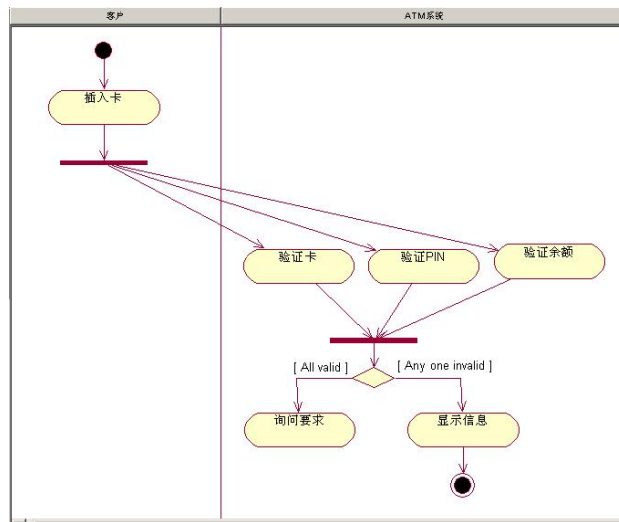


图

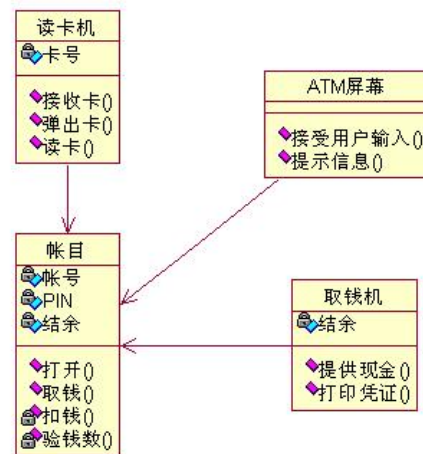
用例图



活动图



类图

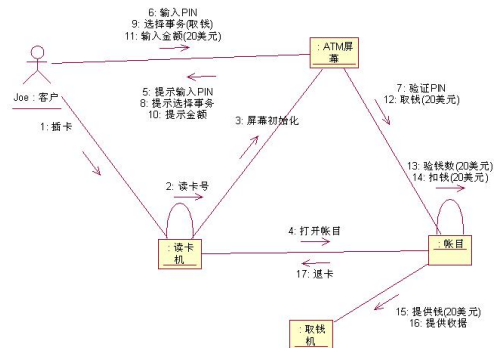
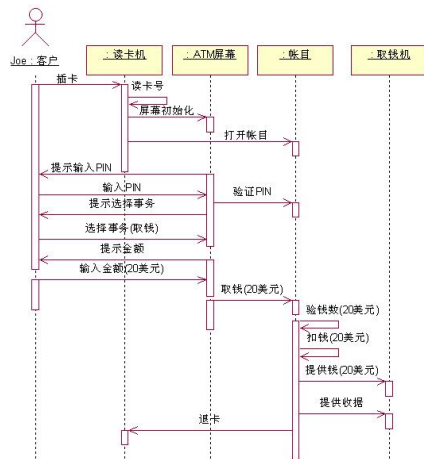
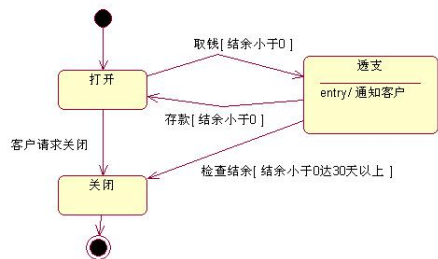


图

序列图

协作图

状态图





UML中类图关系



面向对象方法与UML

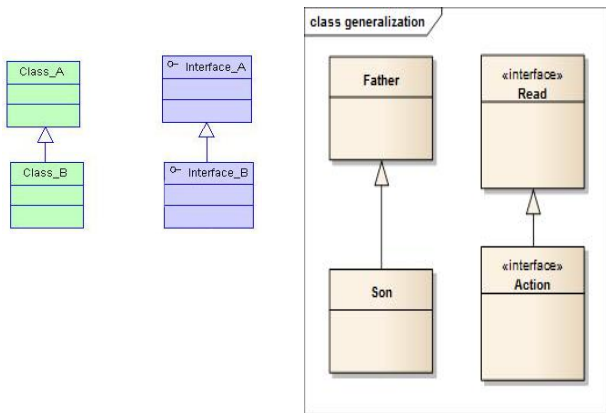
UML的关系



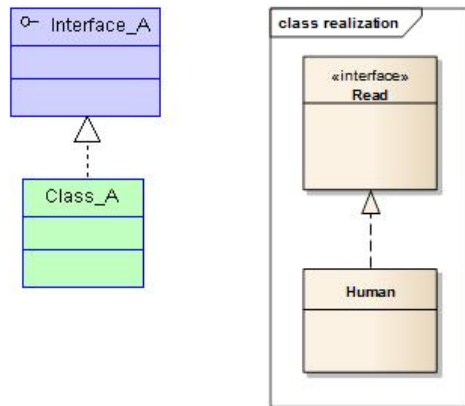
面向对象方法与UML

UML的关系

泛化(generalization)关系是一个类（称为子类、子接口）继承另外的一个类（称为父类、父接口）的功能，并可以增加它自己的新功能的能力，继承是类与类或者接口与接口之间最常见的关系；在Java中此类关系通过关键字`extends`明确标识



实现(realization)关系指的是一个class类实现interface接口（可以是多个）的功能；实现是类与接口之间最常见的关系；在Java中此类关系通过关键字`implements`明确标识

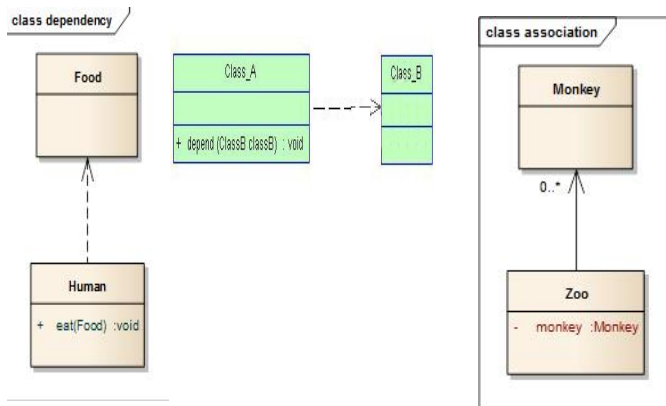


面向对象方法与UML

UML的关系

依赖(dependency)关系:也是类与类之间的连接. 表示一个类依赖于另一个类的定义. 依赖关系总是**单向**的。可以简单的理解, 就是一个类A使用到了另一个类B, 而这种使用关系是具有偶然性的、临时性的、非常弱的, 但是B类的变化会影响到A; 比如某人要过河, 需要借用一条船, 此时人与船之间的关系就是依赖; 表现在代码层面, 为类B作为参数被类A在某个method方法中使用。

在java 中. 依赖关系体现为: **局部变量, 方法中的参数, 和对静态方法的调用。**



关联(association)关系:表示类与类之间的联接, **它使一个类知道另一个类的属性和方法。**

关联可以使用单箭头表示单向关联, 使用双箭头或不使用箭头表示双向关联, 不建议使用双向关联. 关联有两个端点, 在每个端点可以有一个基数, 表示这个关联的类可以有几个实例.

常见的基数及含义:

0..1:0 或1 个实例.

0..*: 对实例的数目没有限制.

1: 只能有一个实例.

1..*: 至少有一个实例.

他体现的是两个类、或者类与接口之间语义级别的一种强依赖关系, 比如我和我的朋友; 这种关系比依赖更强, 不存在依赖关系的偶然性、关系也不是临时性的, 一般是长期性的, 而且双方的关系一般是平等的, 表现在代码层面, 为被关联类B以类属性的形式出现在关联类A中, 也可能是关联类A引用了一个类型为被关联类B的全局变量; 在java 语言中关联关系是使用实例变量实现的。



面向对象方法与UML

UML的关系

聚合(aggregation)关系: 关联关系的一种特例, 是强的关联关系. 聚合是整体和个体之间的关系, 即 **has-a** 的关系, 此时整体与部分之间是可分离的, **他们可以具有各自的生命周期**, 部分可以属于多个整体对象, 也可以为多个整体对象共享; 比如计算机与CPU、公司与员工的关系等;

关联关系中两个类是处于相同的层次, 而聚合关系中两不类是处于不平等的层次, 一个表示整体, 一个表示部分.



组合(合成)关系(composition): 也是关联关系的一种特例, 他体现的是一种 **contains-a** 的关系, 这种关系比聚合更强, 也称为强聚合; 他同样体现整体与部分间的关系, 但此时整体与部分是**不可分的**, **整体的生命周期结束也就意味着部分的生命周期结束**; 比如你和你的大脑; **合成关系不能共享**。

组合跟聚合几乎相同, 唯一的区别就是“部分”不能脱离“整体”单独存在, 就是说, “部分”的生命周期不能比“整体”还要长。



面向对象方法与UML

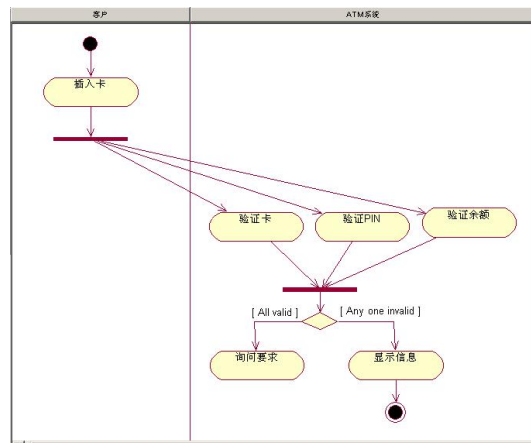
UML的关系

总结：

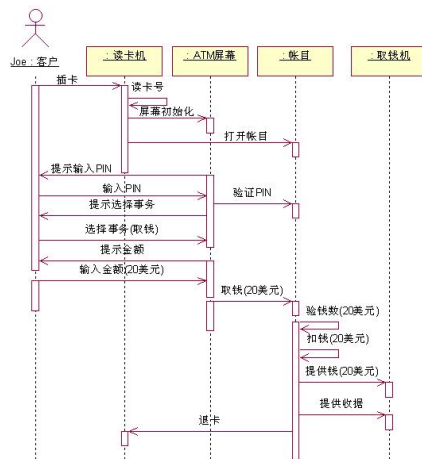
对于继承、实现这两种关系没多少疑问，他们体现的是一种类与类、或者类与接口间的纵向关系；其他的四者关系则体现的是类与类、或者类与接口间的引用、横向关系，是比较难区分的，有很多事物间的关系要想准备定位是很难的，前面也提到，这几种关系都是语义级别的，所以从代码层面并不能完全区分各种关系；但总的来说，后几种关系所表现的强弱程度依次为：**组合>聚合>关联>依赖**。

图

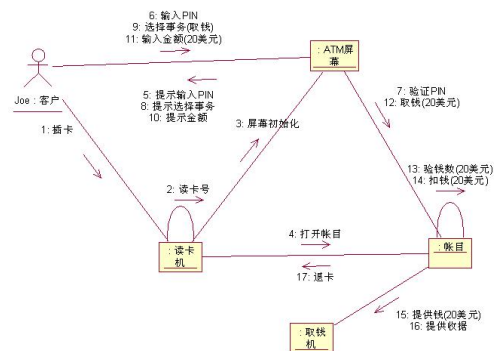
活动图



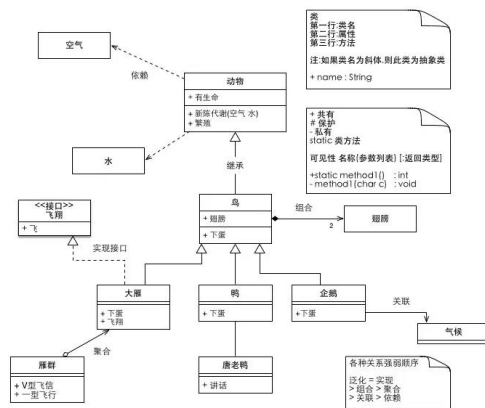
序列图



协作图



UML关系



泛化 (Generalization)



【泛化关系】：是一种继承关系，表示一般与特殊的关系，它指定了子类如何泛化父类的所有特征和行为。例如：老虎是动物的一种，即有老虎的特性也有动物的共性。

【箭头指向】：带三角箭头的实线，箭头指向父类



实现 (Realization) <http://blog.csdn.net/>

【实现关系】：是一种类与接口的关系，表示类是接口所有特征和行为的实现。

【箭头指向】：带三角箭头的虚线，箭头指向接口

组合 (Composition)

【组合关系】：是整体与部分的关系，但部分不能离开整体而单独存在，如公司和部门是整体与部分的关系，没有公司就不存在部门。

【代码体现】：成员变量
【箭头及指向】：带实心菱形的实线，菱形指向整体

聚合 (Aggregation)

【聚合关系】：是整体与部分的关系，且部分可以离开整体而单独存在，如车和轮胎是整体与部分的关系，轮胎离开车仍然可以存在。

【代码体现】：成员变量
【箭头及指向】：带空心菱形的实线，菱形指向整体

UML关系

首先是类：在UML中，我们用分成三栏的长方形表示。第一栏是名称，第二栏是属性，第三栏是方法。

继承：唐老鸭属于鸭的一种，或者说是鸭的细化划分，而鸭属于鸟类，鸟类属于动物，这里无论表达成“是”还是属于，就是这样一个直观的关系。用实线一端带空心三角箭头表示。箭头指向父类。

依赖：凡是动物，生存都需要水和空气，这种必须的需求，我们称之为依赖关系。用虚线一端带箭头表示，箭头指向依赖物。

UML关系

聚合、组合：这两个关系我们放在一起，因为她们看似十分的相似，但是却有着本质的区别。

很多大雁聚合在一起变成雁群，也可以说雁群里有雁，所以是整体和个体之间的关系,即has-a的关系，此时整体与部分之间是可分离的，他们可以具有各自的生命周期。即雁群可以不存在了，但是孤雁仍然存在。

一只鸟由两个翅膀组合而成，即一只鸟有两个翅膀。虽然组合和聚合我们都说“有”什么，但是这个“有”是不同的，他体现的是一种contains-a的关系，这种关系比聚合更强，也称为强聚合；他同样体现整体与部分间的关系，但此时整体与部分是不可分的，整体的生命周期结束也就意味着部分的生命周期结束，比如这只鸟死了，自然这两个翅膀也就不存在了。

UML关系

关联：企鹅和气候有关联，关联是最常见的关系，表示类与类之间的联接，它使一个类知道另一个类的属性和方法。关联有两个端点，在每个端点可以有一个基数，表示这个关联的类可以有几个实例。

常见的基数及含义：0..1:0 或1 个实例。

0..*: 对实例的数目没有限制。

1: 只能有一个实例。

1..*: 至少有一个实例。

The background features a dark, geometric pattern of overlapping triangles in shades of blue and grey. A large, bright cyan hexagon is centered in the upper half of the image. Below the hexagon, two thin, wavy lines in light blue and light green cross the frame horizontally. The bottom half of the image is a solid, dark teal color.

设计模式

设计模式



设计模式



**未调节功能点UFP和
交付功能点DFP**

软件项目估算

1. 已知某银行信息管理系统的规格说明如下:该银行系统能增加新客户,并能从客户文件中做删除。系统支持客户的存款和提款业务。在出现透支时,系统应给出警告信息。客户课通过终端查询自己的账户余额,可以要求给出透支客户报告。输入、输出、外部查询、内部文件和接口的参数值为:

假定输入、输出、外部查询、内部文件和接口的个数为:

功能度 \ 复杂性	简单	中等	复杂
外部输入	3	4	6
外部输出	4	5	7
内部逻辑	2	10	15
外部接口	1	7	10
外部查询	3	4	6

功能度 \ 复杂性	简单	中等	复杂
外部输入	1	0	0
外部输出	2	0	0
内部逻辑	3	0	0
外部接口	1	0	0
外部查询	1	0	0

各个影响因子的影响级如下表所示:

因子	因子值	因子	因子值
备份和恢复	2	主文件联机更新	2
数据通信	1	信息域值复杂度	1
分布式处理	3	内部处理复杂度	0
关键性能	1	设计可复用的代码	0
现有的操作环境	3	设计中的转换域安装	1
联机数据输入	2	多次安装	0
多屏幕输入切换	4	易于变更的应用设计	0

在上面信息的基础上,请采用功能点方法,计算未调节功能点UFP和交付功能点DFP的值。

软件项目估算

未调节功能点 $UFP = 3*1 + 4*2 + 2*3 + 1*1 + 3*1 = 21$

影响度 $N = 2 + 1 + 3 + 1 + 3 + 2 + 4 + 2 + 1 + 1 = 20$

复杂度调节因子 $CAF = 0.65 + 0.01N = 0.65 + 0.01*20 = 0.13$

交付功能点 $DFP = CAF \times UFP = 0.13*21 = 2.73$



挣值统计



挣值统计

- 25.12 假设你是一个软件项目管理者，受命为一个小型软件项目进行挣值统计。这个项目共计划了 56 个工作任务，估计需要 582 人日才能完成。目前有 12 个工作任务已经完成，但是，按照项目进度，现在应该完成 15 个任务，下面给出相关进度安排数据（单位：人日），请你做出挣值分析。

任务	计划工作量	实际工作量	任务	计划工作量	实际工作量
1	12.0	12.5	9	12.0	10.0
2	15.0	11.0	10	6.0	6.5
3	13.0	17.0	11	5.0	4.0
4	8.0	9.5	12	14.0	14.5
5	9.5	9.0	13	16.0	—
6	18.0	19.0	14	6.0	—
7	10.0	10.0	15	8.0	—
8	4.0	4.5			

计算该项目的进度表执行指标 SPI、进度表偏差 SV、预定完成百分比、完成百分比、成本执行指标 CPI、成本偏差 CV。

挣值统计

挣值：已经完成工作的预算成本。

完成所有工作的预算成本：BAC=582人日

在目前的时间点情况下：

预计工作的预算成本：BCWS=156.5人日（总计划工作量）

已完成工作的预算成本：BCWP=126.5人日（已完成计划工作量）

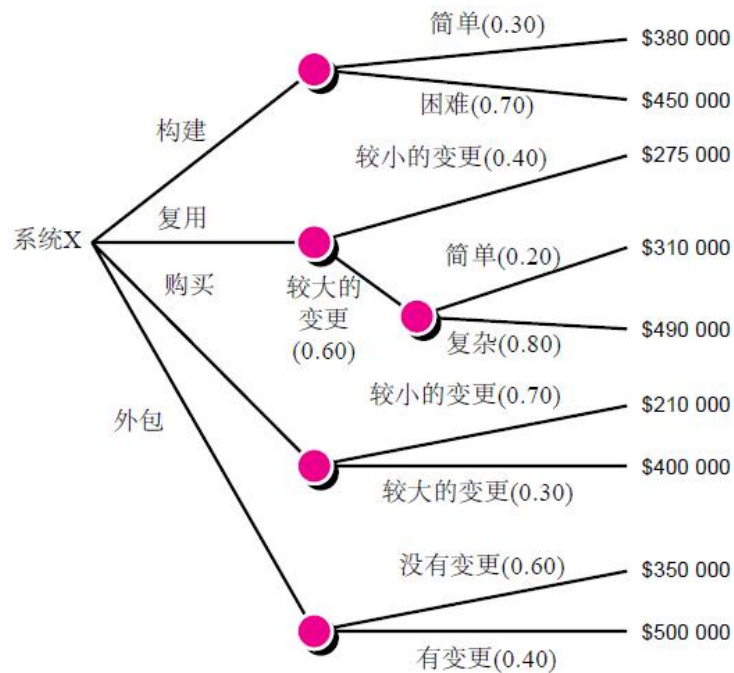
已完成工作的实际成本：ACWP=127.5人日（已完成实际工作量）

1. 进度表执行指标（已完成计划工作量/总计划工作量） $SPI = BCWP/BCWS = 126.5/156.5 = 0.808$ （80.8%）
2. 进度表偏差（已完成计划工作量 - 总计划工作量） $SV = BCWP - BCWS = 126.5 - 156.5 = -30$ 人日
3. 预定完成百分比 = $BCWS/BAC = 156.5/582 = 0.269$ （26.9%）（总计划工作量/完成所有工作的预算成本）
4. 完成百分比 = $BCWP/BAC = 126.5/582 = 0.217$ （21.7%）（已完成计划工作量/完成所有工作的预算成本）
5. 成本执行指标 $CPI = BCWP/ACWP = 126.5/127.5 = 0.992$ （99.2%）（已完成计划工作量 / 已完成实际工作量）
6. 成本偏差 $CV = BCWP - ACWP = 126.5 - 127.5 = -1$ 人日（已完成计划工作量 - 已完成实际工作量）



**自行开发或购买的
决策**

自行开发或购买的决策



计算预期成本

预期成本 =

$$\sum (\text{路径概率})_i \times (\text{估算的路径成本})_i$$

例如，构造系统的预期成本是：

$$\begin{aligned}\text{预期成本}_{\text{构造}} &= 0.30 (\$380\text{K}) + 0.70 (\$450\text{K}) \\ &= \$429 \text{ K}\end{aligned}$$

类似的，

$$\text{预期成本}_{\text{复用}} = \$382\text{K}$$

$$\text{预期成本}_{\text{购买}} = \$267\text{K}$$

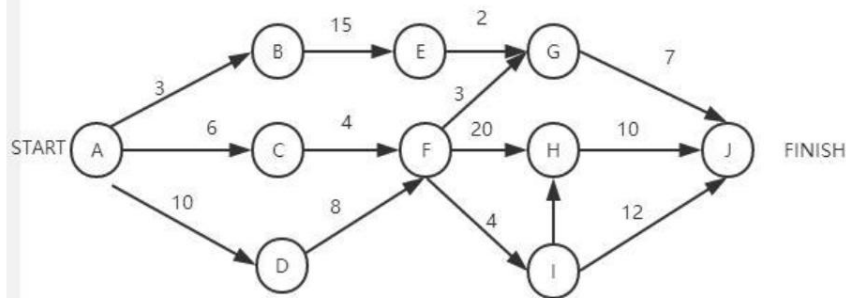
$$\text{预期成本}_{\text{外包}} = \$410\text{K}$$



Pert圖

Pert图（工程网络图）

用网络图来表达项目中各项活动的进度和它们之间的相互关系，在此基础上，进行网络分析和时间估计。



求出工程的最少时间，即关键路径。

首先计算出各个路径长度：

1. $ABEGJ: 3+15+2+7=27$

2. $ACFGJ: 6+4+3+7=20$

3. $ACFHJ: 6+4+20+10=40$

4. $ADFGJ: 10+8+3+7=28$

5. $ADFHJ: 10+8+20+10=48$

6. $ADFIHJ: 10+8+4+10=32$

7. $ADFIJ: 10+8+4+12=34$

综上最长为48，故最少时间为48

谢 谢

